M4: личная корзина покупателя

Вход: каталог, размеры и JWT уже работают. Теперь связываем пользователя и выбранный размер.

Метод	Что делает
GET /cart	Показывает свою корзину
POST /cart/items	Добавляет количество
PATCH /cart/items/{id}	Задаёт новое количество
DELETE /cart/items/{id}	Удаляет свою позицию

Три главных правила

Владелец — из JWT.
Покупаем конкретный размер.
Остаток пока не списываем.

Готовность M4

Два пользователя не видят
и не меняют корзины друг друга.

↳ Корзина хранит намерение купить. Наличие товара повторно проверяется при оформлении заказа.

Шаг 1. Добавить именно размер

product_size_id выбирает складскую позицию; user.id определяет владельца.

HTTP · вход от покупателя — Учебный пример

```
POST /cart/items
Authorization: Bearer …

{"product_size_id": 101,
 "quantity": 1}
```

Сервер проверяет

JWT → User → размер101 → stock.
quantity: строго целое > 0.
user_id в JSON → 422.

Ответ 200

id позиции корзины, размер, цена,
quantity, stock и subtotal.

↳ Товар10 — карточка модели. Размер101 — конкретная строка, у которой есть свой stock.

Шаг 2. Собрать ответ своей корзины

Один запрос связывает CartItem → ProductSize → Product и ограничивает владельца.

SQLAlchemy · фрагмент фильтра — Учебный пример

```
query = select(CartItem,
    ProductSize, Product)
# JOIN размера и товара
query = query.where(
    CartItem.user_id == user_id
).order_by(CartItem.id)
```

Поле ответа	Откуда берём
quantity	CartItem
size, stock	ProductSize
name, price	Product
subtotal	price × quantity

estimated_total = сумма subtotal.
Пустая корзина: 200 · items=[] · 0.00.

↳ Цена в корзине текущая и ориентировочная. В заказе сохраним отдельный снимок цены покупки.

Шаг 3. Проверить границу двух аккаунтов

Аня создала позицию. Смените участника и сравните результат попытки изменить её.

Участник	PATCH / DELETE	Данные
Борис	404 / 404	Без изменений
Администратор	404 / 404	Без изменений

↳ В личных маршрутах роль администратора не заменяет проверку принадлежности позиции.

Шаг 4. Изменить число или удалить строку

PATCH задаёт абсолютное количество. DELETE завершает удаление без тела ответа.

Используйте id из ответа POST — Учебный пример

```
PATCH /cart/items/7
{"quantity": 1}
# Было2 → стало1, ответ200

DELETE /cart/items/7
# Ответ204, тело пустое
```

PATCH

Своя строка → размер под блокировкой
→ новое число не больше stock
→ сохранение.

DELETE

Удаляем только свою строку.
Stock не меняется.
Повторное удаление → 404.

↳ quantity=0 отклоняем с422. Удаление выражено отдельным методом DELETE.

Шаг 5. Повторный POST увеличивает строку

Для одной пары user_id + product_size_id в корзине остаётся одна запись.

Действие	Строк корзины	Количество
Первый POST +1	1	1
Второй POST +1	1	2
Третий POST +2	1	2 · отказ409

Псевдокод · под блокировкой User — Учебный пример

```
item = найти(user_id, size_id)
new_qty = (item.qty if item else 0) + delta
проверить(new_qty <= stock)
создать_или_обновить(item, new_qty)
```

Исходный stock101=3. Во всех трёх случаях stock остаётся3.

↳ UNIQUE запрещает дубликаты. Общая блокировка пользователя упорядочивает чтение и изменение количества.

Шаг 6. Проверить ошибки корзины

Для каждого отказа проверяем, что корзина и остатки сохранились.

Сценарий	Ответ
quantity=0, −1 или true	422
Лишний user_id в JSON	422
Неизвестный размер	404
Чужая позиция корзины	404
Stock=0 или не хватает	409

Как воспроизвести

Зафиксируйте GET /cart и stock.
Отправьте неверный запрос.
Повторите чтение и сравните.

Без токена

Все четыре операции → 401.
Публичность каталога не переносится на личную корзину.

↳ Код отказа и неизменность данных — два отдельных условия успешной проверки.

Принимаем M4 перед оформлением

Пройдите путь двумя аккаунтами и сохраните результаты, которые можно повторить.

Проверяем поведение

Добавить → прочитать → повторить
→ PATCH → DELETE.
Пустая корзина возвращает200.

Проверяем границы

Чужой ID → 404.
Неверное число → 422.
Нехватка → 409, stock прежний.

Эталон · из sneaker-practice — Учебный пример

```
python -m uvicorn app.cart_main:app
# http://127.0.0.1:8000/docs

# Общая проверка M4—M5:
python verify_m45.py
```

↳ Перед M5 ученик объясняет владельца, размер, количество и отсутствие резерва без подсказки.

M5: из корзины создаём один заказ

POST /orders использует текущую корзину, владельца из JWT и цены из PostgreSQL.

Размер	Количество	Цена	Сумма
101	2	7990	15980
201	1	5990	5990
Итого	3 пары	—	21970

HTTP · контракт оформления — Учебный пример

```
POST /orders
Authorization: Bearer …
{}

# Успех:201, status="created"
```

Допустимо и без тела. user_id, total и цены от клиента не принимаются.

↳ Заказ, его позиции, списание остатков и очистка своей корзины должны завершиться вместе.

Шаг 1. Обозначить границу транзакции

Сначала правила и порядок действий. Затем код, повторяющий эту последовательность.

1	**Проверить**	Своя корзина, блокировки, цены и ВСЕ остатки.
2	**Записать**	Order → OrderItem → уменьшить stock → очистить свою корзину.
3	**Завершить**	Один COMMIT; при исключении — ROLLBACK всей операции.

↳ Нельзя зафиксировать заказ отдельно от списания и очистки. Частичная покупка здесь недопустима.

Шаг 2. Проверить все позиции и сумму

Проверка выполняется внутри транзакции, после получения блокировок.

Псевдокод · до первой записи — Учебный пример

```
if cart пустая: ошибка409
for item in cart:
    size = заблокированный_размер(item)
    if item.qty > size.stock: ошибка409
total = sum(price * qty, Decimal("0.00"))
```

Вторая позиция закончилась

101: нужно2, есть3 — подходит.
201: нужно1, есть0 — отказ409.
Первую позицию тоже не покупаем.

Цена — из базы

Decimal, а не float.
Итог должен помещаться в Numeric(12,2).

↳ Добавление в корзину вчера не подтверждает наличие сегодня. Проверяем заново каждую позицию.

Шаг 3. Сохранить заказ и снимки цен

Order описывает покупку целиком. OrderItem фиксирует каждую купленную позицию.

SQLAlchemy · фрагмент внутри транзакции — Учебный пример

```
order = Order(user_id=user_id,
    status="created", total=total)
db.add(order)
db.flush()  # получаем order.id
# Далее для каждой позиции:
db.add(OrderItem(order_id=order.id,
    product_size_id=size.id, quantity=qty,
    price_at_purchase=price))
```

Зачем отдельная цена

Product.price завтра:8990.
price_at_purchase в заказе:7990.
История не дорожает задним числом.

flush ≠ commit

ID уже получен, но изменения
ещё можно полностью откатить.

↳ Историческую сумму считаем по price_at_purchase, а не по сегодняшней цене каталога.

Шаг 4. Списать и очистить вместе

Те же блокировки и та же транзакция охватывают последние изменения.

SQLAlchemy · продолжение сервиса — Учебный пример

```
size.stock -= item.quantity
# Повторяем для всех позиций

db.execute(delete(CartItem).where(
    CartItem.user_id == user_id
))
# commit только на выходе контекста
```

Данные	До	После успеха
Stock101	3	1
Stock201	5	4
Корзина Ани	2 строки	0 строк
Корзина Бориса	1 строка	1 строка

Если сбой: все четыре строки таблицы возвращаются к состоянию «до».

↳ DELETE без условия user_id очистил бы корзины других людей. Эта граница обязательна и внутри транзакции.

Шаг 5. Дать сервису свежую Session

Чтение User при авторизации уже могло автоматически начать транзакцию SQLAlchemy.

Фрагмент · граница сервиса — Учебный пример

```
def checkout(user_id):
    with transaction(owner_id=user_id) as db:
        # новая SessionLocal.begin()
        # проверки + все записи
        result = dto(order)
    return result  # commit уже завершён
```

Авторизация

Своя Session читает User.
В сервис передаём только user.id.

Бизнес-операция

Новая Session → begin → User lock
→ checkout → commit/rollback
→ закрытие Session.

↳ Не добавляйте db.begin() вслепую в сессию, которую уже использовал get_current_user.

16 Практика M5 · заказ и транзакция

Шаг 6. Защитить последнюю пару

Блокировка удерживается до конца транзакции, а проверка stock выполняется после её получения.

SQLAlchemy · выборка размера — Учебный пример

```
size = db.scalar(
    select(ProductSize).where(
        ProductSize.id == size_id
    ).with_for_update()
)
# Только теперь проверяем size.stock
```

Покупатель A	Покупатель B
Получает lock, stock1	Ждёт ту же строку
Списывает1 → commit	Получает lock
201 · заказ создан	Читает stock0 → 409

↳ Общий порядок: User → размеры по id → товары по id. Он уменьшает риск взаимного ожидания.

Шаг 7. Защитить и саму корзину

Одного lock размера мало: два запроса могут работать с корзиной одного человека.

Два checkout Ани

Первый держит User lock и оформляет.
Второй ждёт, видит пустую корзину.
Ответы:201 и409.

Checkout + добавление

Добавление ждёт User lock.
После покупки новая строка301
остаётся в корзине.

Общий договор сервисов — Учебный пример

```
GET /cart          → User lock
POST /cart/items   → User lock
PATCH /cart/items  → User lock
DELETE /cart/items → User lock
POST /orders       → User lock
```

↳ Это защита общей корзины. Полная идемпотентность запроса требует отдельного контракта и хранения ключа.

Шаг 8. Читать только свою историю

Проверка принадлежности нужна и для списка, и для одного заказа по ID.

SQLAlchemy · фрагмент карточки — Учебный пример

```
query = select(Order).where(
    Order.id == order_id,
    Order.user_id == user_id
).options(selectinload(Order.items))
# Нет совпадения → 404
```

Запрос	Контракт
GET /orders	Свои, новые первыми
limit / offset	1…100 / не меньше0
GET /orders/{id}	Своя карточка или404
Нет своих заказов	200 · []

В ответе — total и price_at_purchase.
Роль администратора не открывает чужой GET.

↳ Знание номера заказа не даёт права его читать. Условие user_id остаётся обязательным.

Шаг 9. Менять статус с правом администратора

Учебное правило переходов: created → processing → completed.

HTTP · административная операция — Учебный пример

```
PATCH /orders/8/status
Authorization: Bearer <admin>
{"status": "processing"}

# Повтор того же статуса → 200
# Пропуск или обратный ход → 409
```

Проверяем до записи

require_admin → свежая транзакция → роль из БД → Order FOR UPDATE → допустимый переход.

Склад уже списан

Меняем только status.
Повторного списания и очистки нет.
Неизвестный статус → 422.

↳ Смена статуса описывает обработку существующего заказа, а не новую покупку.

Проверка успеха: считаем результат в БД

Создаём заказ из двух позиций и проверяем все связанные изменения.

Проверка	Фактический результат
HTTP / статус	201 / created
Сумма / позиции	21970.00 / 2
Stock101 / Stock201	1 / 4
Корзина Ани	0 строк
Корзина Бориса	Сохранена

Цены покупки

101:2 × 7990 = 15980
201:1 × 5990 = 5990
total = 21970.00

Что сохранить для ревью

Запрос без секрета, ответ, строки БД до и после, команда проверки.

↳ Одного201 недостаточно: проверяем сумму, снимки цен, остатки и принадлежность очищенной корзины.

Проверка сбоя: отменяем даже поздние записи

Один и тот же ORM-сервис вызывается с контролируемой ошибкой до commit.

Точка сбоя	Заказы	Корзина	Stock101 / 201
После Order	0	2 строки	3 / 5
После списания	0	2 строки	3 / 5
После очистки	0	2 строки	3 / 5

↳ Проверяется совпадение полных снимков таблиц. Эти три проверки вызывают сервис напрямую, без HTTP.

Проверка гонки: два настоящих HTTP-запроса

Тест наблюдает ожидание блокировок, затем освобождает строку и проверяет итог.

Гонка	HTTP	Итог
Последняя пара	201 + 409	1 заказ, stock0
Одна корзина	201 + 409	1 заказ, корзина пуста
Добавление после покупки	201 + 200	301 остаётся в корзине

↳ sleep сам по себе не доказывает гонку. Здесь подтверждено ожидание обоих запросов в PostgreSQL.

Ревью M5: объяснить и доказать покупку

Разберите два отказа прежде, чем открывать ответ. Затем проверьте свой код по промпту.

Полный ROLLBACK

Нет нового заказа и его позиций.
Stock и корзина восстановлены.
Проверяем снимок данных.

Исход нужно выяснить

Сначала сверка истории.
Без Idempotency-Key нельзя
обещать безопасный автоповтор.

↳ 76 проверок M4–M5: HTTP, откат ORM и наблюдаемые гонки. Следующий этап — M6, после проверки главы.